

Engineering Decisions

Architecture Selection

A modular, interface-based architecture was chosen to keep the crawler easy to extend and maintain. The `ProcurementCrawler` interface separates portal-specific scraping logic from the rest of the application, such as the execution flow, database operations, and API layer. This makes it straightforward to support additional procurement portals by implementing a new adapter without changing the core workflow.

The main trade-off is a small increase in initial design complexity, but it provides a clean structure that becomes much easier to maintain as more portals are added.

Why PostgreSQL?

PostgreSQL was selected because it provides reliable ACID transactions, strong indexing support, and JSONB storage for portal-specific data. Since procurement websites often expose slightly different information, combining structured columns with JSONB allows the application to store common fields while still preserving additional raw data when required.

Prisma was used as the ORM because it offers type safety, better developer productivity, and easier schema management. Although raw SQL may provide slightly better performance, Prisma is a practical choice for a proof-of-concept of this size.

Scraping Approach

The selected procurement portal (UK Contracts Finder) serves its content as server-rendered HTML, so a lightweight HTML parser (`cheerio`) together with the native `fetch` API was sufficient.

Compared to browser automation tools such as Playwright, this approach uses fewer system resources and executes much faster for static pages.

One limitation is that HTML selectors may break if the website layout changes. To reduce the impact, all selectors are isolated within the portal adapter, allowing changes to be made in one place without affecting the rest of the application.

Duplicate Handling Strategy

Duplicate tenders are prevented using a composite unique constraint on (`portalName`, `tenderId`).

The crawler uses Prisma's `upsert` operation so repeated crawler runs update existing records instead of creating duplicates. Fields such as title, closing date, tender value, document hash, and `lastSeenAt` are refreshed whenever new information is available.

For significantly larger datasets, batch processing or bulk database operations would be considered to improve throughput.

Document Storage and Hashing

Tender PDFs are downloaded using Node.js streams rather than loading the entire file into memory. This keeps memory usage low and allows the application to handle larger documents efficiently.

After the download completes, a SHA-256 hash is generated to help detect document changes across crawler runs.

For a production deployment, local file storage could easily be replaced with an object storage solution such as Amazon S3.

Error Handling and Logging

`pino` was selected for logging because it is lightweight, fast, and produces structured JSON logs that integrate well with monitoring platforms such as ELK and Datadog.

The API uses centralized Express error-handling middleware to return consistent error responses, while crawler failures are handled on a per-record basis. This ensures that one failed tender does not interrupt the entire crawling process.

As a future improvement, retry logic with exponential backoff could be added for temporary network failures.

Key Trade-offs

- **Cheerio vs Playwright:** Cheerio was chosen because the target website is server-rendered, making browser automation unnecessary while reducing CPU and memory usage.
- **Local Storage vs Cloud Storage:** Local storage keeps the proof-of-concept simple, while cloud object storage would be more suitable for production.
- **Offset Pagination vs Cursor Pagination:** Offset pagination is sufficient for the current dataset. Cursor-based pagination would perform better with very large datasets.
- **Single Database vs Distributed Database:** A single PostgreSQL instance keeps deployment simple. At larger scale, read replicas or database partitioning could be introduced.

Scalability to 5,000+ Websites

The current architecture is designed so it can evolve without major redesign.

As the number of supported procurement portals increases, the platform could be extended by:

- Using message queues such as RabbitMQ or AWS SQS to distribute crawler jobs.
- Running crawler workers across multiple machines or IP addresses to avoid rate limiting.
- Using browser clusters for websites that require JavaScript rendering.
- Moving PDF storage to cloud object storage such as Amazon S3.
- Introducing automated monitoring and AI-assisted parser generation to detect and adapt to website structure changes while keeping engineering review in the loop.

Future Improvements

Some enhancements that would be valuable beyond this proof-of-concept include:

- Incremental crawling based on change detection.

- Distributed scheduling and workload balancing.
- Metrics collection using Prometheus and Grafana.
- Authentication and rate limiting for the REST APIs.
- Automated integration tests against representative procurement portals.